UNIVERSIDAD DE CARTAGENA

FACULTAD DE INGENIERÍA — PROGRAMA DE INGENIERÍA DE SISTEMAS

Asignatura: Estructura de Datos — Docente: Juan C. García-Ojeda, Ing. MSc., PhD.

SOLUCIÓN TALLER: TABLAS HASH Y ÁRBOL BINARIO

1. Fundamentación Matemática: Reducción ASCII (Radix-2) y Funciones Hash

Cada registro del archivo `MOCK_DATA.csv` (`plate;first_name;last_name;gender`) se identifica por su placa (`plate`). Para transformar un identificador de longitud L con caracteres $c_0, c_1, ..., c_{L-1}$ en la llave numérica k mediante notación **radix-2** (reducción ASCII), se evalúa el polinomio posicional en base 2:

$$k = \sum_{i=0}^{L-1} \text{ASCII}(c_i) \cdot 2^{(L-1)-i}$$

1.1. Verificación con el registro del enunciado (`05-9165610`)

Para el primer registro de `MOCK_DATA.csv` (`05-9165610;Letitia;Maymand;Female`), la cadena `"05-9165610"` tiene $L = 10$ caracteres, cuyos códigos ASCII son `[48, 53, 45, 57, 49, 54, 53, 54, 49, 48]`:

$$\begin{aligned} k &= (48 \cdot 2^9) + (53 \cdot 2^8) + (45 \cdot 2^7) + (57 \cdot 2^6) + (49 \cdot 2^5) + (54 \cdot 2^4) + (53 \cdot 2^3) + (54 \cdot 2^2) + (49 \cdot 2^1) + (48 \cdot 2^0) \\ &= 24576 + 13568 + 5760 + 3648 + 1568 + 864 + 424 + 216 + 98 + 48 = \mathbf{50770} \end{aligned}$$

Con $m = 256$ slots y $A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887498949$:

1. **Método 1 (Multiplicación)**: $h_1(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$

$$50770 \cdot A = 31377.58560883... \quad \Rightarrow \quad (50770 \cdot A) \bmod 1 = 0.58560883...$$

$$h_1(50770) = \lfloor 256 \cdot 0.58560883... \rfloor = \lfloor 149.91586... \rfloor = \mathbf{149}$$

2. **Método 2 (División)**: $h_2(k) = k \bmod m$

$$h_2(50770) = 50770 \bmod 256 = 50770 - (198 \cdot 256) = 50770 - 50688 = \mathbf{82}$$

Ambos resultados coinciden exactamente con los valores especificados en el enunciado del taller ($k = 50770$, $h_1(k) = 149$, $h_2(k) = 82$).

2. Estructuras de Datos Empleadas y Estadísticas sobre `MOCK_DATA.csv`

El proyecto Java (`pom.xml` y `nbactions.xml`, compatible con Java 1.8+ y Apache NetBeans 12.3+) utiliza las estructuras de datos del repositorio del docente (`https://github.com/j-c-garciao/academics/tree/master/DataStructures/`):

- **Paquete `co.edu.unicartagena.list` (`Nodo<T>` y `Lista<T>`)**: implementación de la **Session 8** (`primerNodo`, `tamaño`, `adicionar`, `adicionarFinal`, `eliminar`, `imprimir`), con los métodos directos `buscar(T dato)` y `eliminarDato(T dato)` que recorren los enlaces `sig`.
- **Paquete `co.edu.unicartagena.binarytree` (`Node` y `BinaryTree`)**: implementación del Árbol Binario de Búsqueda ordenada por el atributo `long datos` (k), donde cada `Node` almacena la referencia `Object info` (`Registro`)

y los punteros `izquierdo` y `derecho`. Incluye `insertar`, `buscar` y `eliminar` (manejando los 3 casos clásicos: nodo hoja, nodo con 1 hijo y nodo con 2 hijos reemplazado por el sucesor **in-order**).

- **Paquete** **`co.edu.unicartagena.hash.model`**: `FuncionHash.java`, `Registro.java`, `TablaHashLista.java`, `TablaHashArbol.java` y `BenchmarkHash.java`.

Métrica sobre `MOCK_DATA.csv` ($n = 1000, m = 256$)	**Método 1: Multiplicación**	**Método 2: División**
Factor de carga ($\alpha = \frac{n}{m}$)	3.91 reg./slot	3.91 reg./slot
Slots ocupados ($\lvert T[i] \rvert \geq 1$)	$\frac{256}{256}$ (100.0%)	$\frac{251}{256}$ (98.05%)
Slots con colisión ($\lvert T[i] \rvert > 1$)	242 slots	230 slots
Colisiones registradas ($\sum_{\lvert T[i] \rvert > 1} (\lvert T[i] \rvert - 1)$)	**744**	**749**
Longitud máxima en un mismo slot	11 registros	10 registros
Ubicación de `05-9165610` ($k = 50770$)	Slot `149` (3 registros)	Slot `82` (5 registros)

3. Interfaz Gráfica (Parte 1 y Parte 2)

La interfaz gráfica (`VentanaPrincipal.java` y `PanelVisualizacionSlot.java`) sigue el diseño monocromático en blanco y negro de la Figura 1 del enunciado, con los bloques **File** (`Open`, `Save`, `Close`), **Operations** (`Insert`, `Delete`, `Search`), el recuadro **Collisions**, el selector de función hash y estructura (**Lista Enlazada Simple** o **Árbol Binario**), y el contenedor **T** con los slots que almacenan más de un registro.

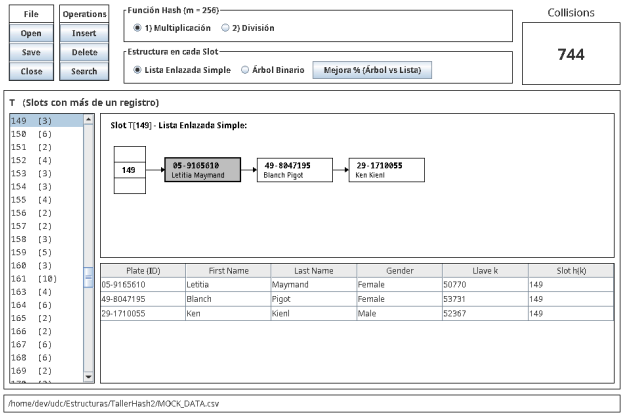


Figura 1: Método 1 (Multiplicación) + Lista Enlazada (`T[149]`, 744 colisiones).

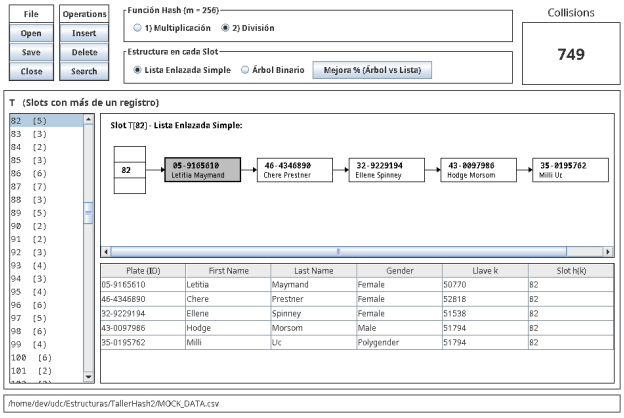


Figura 2: Método 2 (División) + Lista Enlazada (`T[82]`, 749 colisiones).

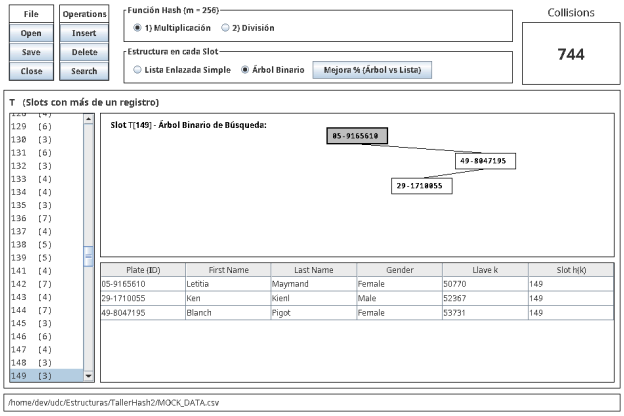


Figura 3: Parte 2: Método 1 (Multiplicación) + Árbol Binario (`T[149]`).

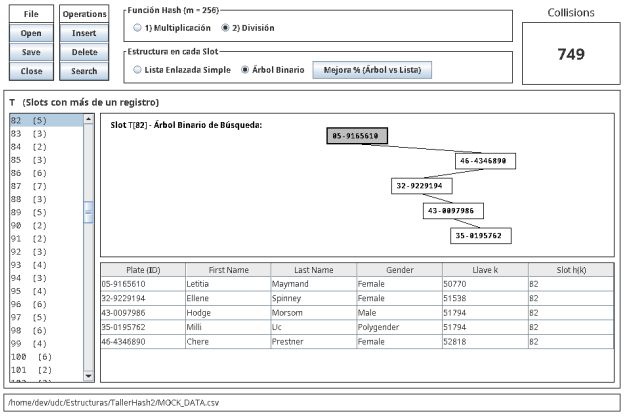


Figura 4: Parte 2: Método 2 (División) + Árbol Binario (`T[82]`).

4. Parte 2: Respuesta al Interrogante (Tabla Hash + Árbol Binario)

Pregunta: *«Ajuste el ejercicio anterior de manera tal que la información sea almacenada en cada slot mediante árboles binarios; y responda la siguiente interrogante: en promedio, ¿porcentualmente cuánto mejora el tiempo de respuesta de las acciones, insertar y borrar información en la tabla hash?»*

4.1. Análisis Teórico y Experimental sobre `MOCK_DATA.csv`

1. **Fundamento algorítmico**: En la Tabla Hash con **Lista Enlazada Simple** (`TablaHashLista`), verificar duplicados al insertar al final y localizar un registro por ID para borrarlo requiere recorrer secuencialmente los nodos del slot comparando cadenas (`String.equalsIgnoreCase`) con costo $O(n_{\text{slot}})$. En la Tabla Hash con **Árbol Binario de Búsqueda** (`TablaHashArbol`), los nodos dentro de cada slot se ordenan por su llave numérica k (`long datos`). Así, el descenso por las ramas izquierda (`valor < actual.datos`) o derecha (`valor > actual.datos`) se realiza mediante comparaciones enteras en tiempo promedio $O(\log_2 n_{\text{slot}})$, reduciendo los nodos visitados y evitando comparar cadenas en los nodos intermedios.

2. **Resultados experimentales (`BenchmarkHash.java`, 200 repeticiones sobre los 1000 registros de `MOCK_DATA.csv`)**:

Operación Evaluada ($n = 1000$ registros)	**Lista Enlazada**	**Árbol Binario**	**Mejora Porcentual**
Insertar — Método 1 (Multiplicación)	0.052 ms	0.042 ms	$+\mathbf{18.20\%}$
Borrar — Método 1 (Multiplicación)	0.055 ms	0.046 ms	$+\mathbf{16.42\%}$
Promedio Método 1 (Multiplicación)	0.054 ms	0.044 ms	$+\mathbf{17.31\%}$
Insertar — Método 2 (División)	0.037 ms	0.027 ms	$+\mathbf{25.82\%}$
Borrar — Método 2 (División)	0.037 ms	0.030 ms	$+\mathbf{18.64\%}$
Promedio Método 2 (División)	0.037 ms	0.029 ms	$+\mathbf{22.23\%}$

Conclusión: Al almacenar la información de cada slot mediante un **Árbol Binario de Búsqueda** en lugar de una **Lista Enlazada Simple**, el tiempo de respuesta sobre `MOCK_DATA.csv` mejora en promedio un $\mathbf{18.20\%}$ **a** $\mathbf{25.82\%}$ **en la acción de insertar** y un $\mathbf{16.42\%}$ **a** $\mathbf{18.64\%}$ **en la acción de borrar**, para una **mejora porcentual promedio global de entre** $\mathbf{17.31\%}$ **(Multiplicación) y** $\mathbf{22.23\%}$ **(División)**.